

Rendimiento: conteo de palabras secuencial vs. paralelo

Adrián González (23152), Fernando Mendoza (19644), Hansel López (19026)

Computación Paralela y Distribuida

Se compararon la versión secuencial y la versión paralela del programa de conteo de frecuencia de palabras usado en el Corto 02. Ambos programas ya median el tiempo con `std::chrono`, así que solo se corrió el benchmark: 5 ejecuciones con el archivo `grande.txt`, para secuencial, paralelo con 2 hilos y paralelo con 4 hilos. El tiempo medido es solo el del conteo; en paralelo incluye crear los hilos, hacer join y combinar los mapas locales.

Tiempos de ejecución (ms)

Num.	Secuencial	Paralela 2 hilos	Paralela 4 hilos
1	7.5092	8.5516	8.7744
2	7.4538	8.3210	8.4623
3	7.2411	8.2867	8.2782
4	7.2394	8.6969	8.4505
5	7.4437	8.1682	8.0784
Promedio	7.3774	8.4049	8.4087

Speedup

$S(p) = \text{tiempo secuencial} / \text{tiempo paralelo}(p)$

Num.	Speedup 2 hilos	Speedup 4 hilos
1	0.8781	0.8558
2	0.8958	0.8808
3	0.8738	0.8747
4	0.8324	0.8567
5	0.9113	0.9214
Con promedios	0.8778	0.8774

Efficiency

$E(p) = S(p) / p$

Num.	Efficiency 2 hilos	Efficiency 4 hilos
1	43.91%	21.40%
2	44.79%	22.02%
3	43.69%	21.87%
4	41.62%	21.42%
5	45.57%	23.04%
Con promedios	43.89%	21.93%

Análisis

Antes que nada, algo importante: estas pruebas se corrieron en una máquina con un solo núcleo disponible. Eso cambia por completo la interpretación de los resultados, así que lo explico primero y luego respondo cada pregunta con eso en mente.

¿Con qué cantidad de hilos se obtuvo el mayor speedup?

Ninguno superó a la versión secuencial. El speedup promedio fue 0.88 con 2 hilos y 0.88 con 4 hilos, prácticamente iguales y ambos por debajo de 1. O sea que las dos versiones paralelas terminaron siendo un poco más lentas que la secuencial, no más rápidas.

¿Con qué configuración se obtuvo la mayor eficiencia?

Con 2 hilos (43.9% contra 21.9% con 4 hilos). Como ninguna configuración logró bajar el tiempo, entre más hilos se usan peor se ve la eficiencia, porque se reparte un speedup que ya de por sí es menor a 1 entre más procesadores.

¿El speedup aumentó proporcionalmente al incrementar los hilos?

No, al contrario. Lo esperado sería acercarse a 2x con 2 hilos y a 4x con 4 hilos; acá se quedó estancado alrededor de 0.88x en ambos casos. Duplicar los hilos no cambió casi nada el tiempo total.

¿La eficiencia aumentó o disminuyó al usar más recursos?

Disminuyó bastante: de 43.9% con 2 hilos a 21.9% con 4 hilos, prácticamente a la mitad. Cada hilo que se agrega diluye más el (poco) beneficio que hay.

¿Qué factores pudieron afectar el tiempo de ejecución?

- El más determinante: la máquina donde se corrió el benchmark solo tiene un núcleo físico disponible. Sin núcleos extra, los hilos no corren en paralelo de verdad, solo se van turnando en el mismo núcleo por planificación del sistema operativo.
- El tamaño de la tarea es muy pequeño para paralelizar: contar 300,000 palabras toma entre 7 y 8 ms. En ese rango de tiempo, cualquier overhead de crear hilos pesa proporcionalmente mucho.
- Variación normal entre corridas por carga del sistema, caché y el propio scheduler.

¿Qué fuentes de overhead se identifican en el programa?

- Crear y unir (join) los hilos en cada ejecución.
- Reservar un unordered_map local por hilo.
- La fase de reduce: recorrer los mapas locales y sumarlos en el mapa global al final.
- Cambios de contexto del sistema operativo al simular concurrencia sobre un solo núcleo.

¿Agregar más hilos garantiza mejor rendimiento?

No, y estos resultados lo muestran de forma bastante clara: pasar de 2 a 4 hilos no bajó el tiempo, subió la eficiencia perdida. El paralelismo solo ayuda cuando hay núcleos físicos de sobra para repartir el trabajo y cuando la tarea es lo bastante grande para que ese trabajo compense el overhead de crear y sincronizar hilos. Aquí faltaron las dos cosas: un solo núcleo real y una tarea que se resuelve en milisegundos.

Lo que falta para tener una medición representativa es correr el mismo benchmark.sh en una máquina con varios núcleos físicos (por ejemplo la laptop del equipo o un servidor del lab), y si se quiere ver el paralelismo brillar más, probar también con un archivo bastante más grande que grande.txt, donde el tiempo de conteo domine claramente sobre el costo de crear los hilos.